CSS

Block Positioning

floating blocks, position

doc. Ing. Radek Burget, Ph.D.
burgetr@vut.cz
Ing. Jiří Hynek, Ph.D.
hynek@vut.cz

Layout Modes

- Normal flow
  - So called in-flow elements
  - Elements are laid out in the document order
  - Inline elements on the lines, block elements below each other
- Floating blocks
  - Elements moved to left or right side
  - The `float` property
- Positioned elements
  - The element position is specified explicitly
  - The `position` property

Floating blocks

- Object that are flowed by the surrounding text
- Creating a floating object:
  - `float: left` or
  - `float: right`
- Floating elements become automaticaly **block** elements
- The floating object is
  - Removed from the normal text flow (doesn't influence the text on the original place)
  - Shifted to the border of the parent element
- Different way of width computation
  - It is better to set an explicit width
  - If not set, it is automaticaly computed (shring-to-fit algorithm)

Floating demonstration

- Normal flow (default)

```
…
<div class="floating" id="menu">
    <h1>Menu</h1>
    <ul>
        <li>Red</li>
        <li>Green</li>
        <li>Magenta</li>
        <li>Yellow</li>
        <li>Dark blue</li>
    </ul>
</div>
…
```

Creating a floating object

```
<style type="text/css">
    #menu { background-color: #eef;
            border: 1px solid #aaf;
            width: 12em; }
    .floating { float: left; }
</style>

“` [Sample](data/float1.html)
```

Margins

```
<style type="text/css">
    #menu { background-color: #eef;
            border: 1px solid #aaf;
            width: 12em;
            padding: 0;
            margin-right: 1em; }
    .floating { float: left; }
</style>
```

Heading

```
<style type="text/css">
    #menu { … }
    #menu h1 { font-weight: bold;
               font-size: 100%;
               color: #fff;
               background-color: #55f;
               margin: 0;
               padding: 0.5em; }
    .floating { float: left; }
</style>

“` [Sample](data/float2.html)
```

Paragraph block size

```
<style type="text/css">
    #menu { … }
    #menu h1 { font-weight: bold;
               font-size: 100%;
               color: #fff;
               background-color: #55f;
               margin: 0;
               padding: 0.5em; }
    .plovouci { float: left; }
    p { border: 5px solid red; }
</style>
“` [Sample](data/float3.html)
```

Clear

- The floating objects are placed by the browser
  - Side by side to the appropriate margin
  - When there is no space then below each other
  - => The floating object can horizontaly overflow the flowing text
- When some text shouldn't flow along the floating objects:
  - `clear: left | right | both`
- First, previous floating object are placed, then the text is placed below
- Example

Containing block height (again)

- When a block has `height:auto`, its height depends on its contents
- For `overflow: visible`
  - Only in-flow content is considered
  - If the block contains floating blocks only, it has a **zero height** in the result
- Otherwise (e.g. `overflow: hidden`)
  - Floating content is considered too
- Example

Block Height: Example

```html
<div class="contents">
    <div class="col">Column 1</div>
    <div class="col">Column 2</div>
</div>
<p>Next contents</p>
```

```css
.contents {
    border: 5px solid blue;
}
.col {
    float: left;
    background: gray;
}
```

Block Height: Example (II)

```
.contents {
    overflow: visible;
}
```

Column 1

Column 2

Next contents

Výška bloku: příklad (III)

```
.contents {
    overflow: hidden;
}
```

Column 1

Column 2

Next contents

Positioning

- Manual specification of the object's position on the page
- Absolute
  - Placing an object to a particular place independently on the remaining text
- Relative
  - Moving the object relatively to its normal position
- The `position` property
  - `position: static` – normal flow
  - `position: absolute` – absolute positioning
  - `position: relative` – relative positioning
  - `position: fixed` – fixed positioning

Relative positioning

- The position relative to normal placement
  - `position: relative`
- The block remains in normal flow; it's just moved
- The distance from the top left corner
  - `top: length` - moving down
  - `left: length` - moving right
- The distance from the bottom right corner:
  - `bottom: length` - moving up
  - `right: length` - moving left
- May be combined
- Negative value - reverse direction

Relative positioning - example

```
<style>
    .word {
        font-weight: bold;
    }
</style>
…
This is <span class="word">bold</span> text.
```

This is bold text.

Relative positioning - example

```
<style>
    .word {
        font-weight: bold;
        position: relative;
        top: -0.5em;
        left: 5px;
    }
</style>
…
This is <span class="word">bold</span> text.
```

This is bold text.

Absolute positioning

- `position: absolute`
- The object is **removed** from the normal flow
  - Doesn't influence the text in the normal place
- It is placed to a particular place on the page
  - It can overlap the objects being already on that place
  - It is necessary to „make space”
- The distance from the top left corner of the **containing block**
  - `top: length`
  - `left: length`
- The distance from the bottom left corner
  - `bottom: length`
  - `right: length`

Containing Block

- Initial containing block is the Viewport
- Elements with position `relative` or `static`
  - The nearest ancestor block is the containing block
- Elements with position `absolute`
  - The nearest ancestor block with position `relative`, `absolute` or `fixed` is the containing block
- Elements with position `fixed`
  - Viewport is always the containing block

Coordinate System

- Normaly, the coordinate system is formed by the **Viewport**
- Objects that have `position: absolute` or `relative` or `fixed`, create a **local coordinate system** for absolute positioning
  - Nested object with `position: absolute` are positioned from within the parent

Coordinate System (II)

- Child blocks are positioned within **Viewport** (default): “`html

…

Coordinate System (III)

- Child blocks are positioned within **parent block** : “`html

…

Absolute positioning - example

```
<style>
    .word {
        font-weight: bold;
        position: absolute;
        top: 150px;
        left: 500px;
    }
</style>
…
This is <span class="word">bold</span> text.
```

This is bold text.

Fixed positioning

```
<style>
    .word {
        font-weight: bold;
        position: fixed;
        top: 150px;
        left: 500px;
    }
</style>
…
This is <span class="word">bold</span> text.
```

This is bold text.

Positioned Element Width and Height

- The equation for width:
  left + margin-left + border-left-width +
  + padding-left + width + padding-right +
  + border-right-width + margin-right + **right** = containing_block_width
- The `auto` values for `left` and `top` are derived from static position
- Computation of multiple `auto` values is defined in the specification
- The **height** is computed analogously

Object overlaping

- Two elements may overlap (relative or absolute positioning)
- Normally, the later specified object overlaps the earlier object
- This can be changed using the `z-index` property
  - `z-index: 10`
  - The number gives the Z coordinate of the object (the higher number the „higher” placed is the object)
  - If not specified, **0** is used

Overlaping: example

```
<div style="height: 4em; position: relative">
<div ="" class="box" style="position: absolute;     top: 30px; left: 180px">
       First box</div>
  <div ="" class="box" style="position: absolute; top: 60px; left: 200px">
       Second box</div>
</div>
```

First box

Second box

Overlaping: example

```
<div style="height: 4em; position: relative">
<div ="" class="box" style="position: absolute;     top: 30px; left: 180px; z-index: 1">
       First box</div>
  <div ="" class="box" style="position: absolute; top: 60px; left: 200px">
       Second box</div>
</div>
```

First box

Second box

Object hiding

- Two possibilities
  1. `display: none`
     - The element is totally ignored during rendering
  2. `visibility: hidden`
     - The object occupies the place on the page but it is not displayed

Visibility - example

```
This is <span style="…">invisible</span> text.
```

1. `display: none`

This is invisible text.

2. `visibility: hidden`

This is invisible text.

Why to hide the text

1. `display: none`
   - The elements displayed in browsers with no CSS support (alternative navigation, auxiliary labels, …)
   - Element intended for other media types (e.g. for printing)
2. `visibility: hidden`
   - The elements that may be displayed later (scripting)

Table style

- We can change the style of
  - The whole table - `<table>`
  - Rows - `<tr>`
  - Cells - `<td>`, `<th>`
  - Columns - `<col>`, `<colgroup>`
  - Header, footer, body - `<thead>`, `<tfoot>`, `<tbody>`
  - Caption - `<caption>`

Selectors

- Usually context selectors
  - `table { … }`
  - `table td { … }`
  - `table.results td { … }`
  - `table th.main { … }`
  - `table tr#row1 { … }`
- Using cascade:
  - `tr { … }`
  - `table.results tr { … }`

Table captions - `<caption>`

- Caption position
  - `caption-side: top`
  - `caption-side: bottom`
  - `caption-side: left`
  - `caption-side: right`
- Other properties
  - `text-align`
  - `vertical-align`
  - `width`

Table caption – example

Tab 1. Caption on the top Name | Surname

|---
John | Wayne
Albert | Einstein
Tab 2. Caption in the bottom Name | Surname

|---
John | Wayne
Albert | Einstein

Borders in a table

- Borders can be set separately for the cells or the whole table
- Special properties:
  - `border-collapse: separate | collapse` - cell border merging
  - `border-spacing: 5px` - cell border spacing
  - `empty-cells: show | hide` - whether to diplay or not the empty cells

Examples

```
table { border-collapse: separate }
```

Name | Surname

|---
John | Wayne
Albert | Einstein
`table { border-collapse: collapse }` Name | Surname

|---
John | Wayne
Albert | Einstein

Examples

```
table tr { border-top: 1px solid }
```

Name	Surname

|---
John | Wayne
Albert | Einstein
`table tr { border: 1px solid }` Name | Surname

|---
John | Wayne
Albert | Einstein

Examples

```
@CSS@ table { border: 1px solid } table th { border: 1px solid } table td {
border: none; border-left: 1px solid; } @/CSS@ Name | Surname
```

|---
John | Wayne
Albert | Einstein

Table and cell background

- Each layer is either transparant (`border-color: transparent`) or it has a colour or an image assigned
- The layers go in following order (bottom to top)
  1. Table `<table>`
  2. Column groups `<colgroup>`
  3. Columns `<col>`
  4. Row groups `<thead>`, `<tbody>`, `<tfoot>`
  5. Rows `<tr>`
  6. Cells `<td>`, `<th>`

List styling

- The list items can be style (ordered or unordered)
- We can define
  - **Bullet type** (unordered)
  - **Numbering type** (ordered)
  - **An image instead of the bullet**
  - **Bullet position** (inside or outside)

Bullet type

- The `list-style-type` property
- Applied to the `<li>` element

```
ul li { list-style-type: square; }
```

- Unordered lists:
  - `list-style-type: disc`
  - `list-style-type: circle`
  - `list-style-type: square`

Numbering type

- The `list-style-type` property
- Ordered lists:
  - `list-style-type: decimal`
  - `list-style-type: lower-roman`
  - `list-style-type: upper-roman`
  - `list-style-type: lower-alpha`
  - `list-style-type: upper-alpha`
  - `list-style-type: none`

Images instead of the bullets

- The `list-style-image` property

```
<ul>
<li style="list-style-image: url('ok.gif')">
List item</li>
</ul>
```

- List item

Bullet position

- The `list-style-position` property
- `list-style position: outside`
  - First item in longer text in longer text in longer text in longer text in longer text…
  - First item in longer text in longer text in longer text in longer text in longer text…
- `list-style position: inside`
  - First item in longer text in longer text in longer text in longer text in longer text…
  - First item in longer text in longer text in longer text in longer text in longer text…

The `list-style` property

- Sumarizes the `list-style-*` properties
- Format:
  - `list-style: type position image`
- Examples:
  - `list-style: square inside none`
  - `list-style: lower-alpha outside url("obr.gif")`

Mouse cursor

- The `cursor` property allows to change the mouse cursor look when placed over the particular element on the page.

```
The cursor over this
<span style="cursor: crosshair">word</span>
has a different shape.
```

The cursor over this word has a different shape.

Cursor types

auto| crosshair| default

|---|---
hand| pointer| move
text| wait| help
nw-resize| n-resize| ne-resize

|---|---
w-resize| | e-resize
sw-resize| s-resize| se-resize

- Pointer cursor for all the browsers:
  - `cursor: pointer; cursor: hand`

Pseudo Elements and Classes

- Abstract class for denoting special elements or their state
- They define the look of various parts
  - **Pseudo classes** : in different states
    - E.g. links visited, not visited, focused, etc.
    - Specificity as for classes
  - **Pseudo elements** : different parts of elements
    - First element, first letter, before, after, …
    - Specificity as for elements
- Denoted by „:”
  - `p:first-letter { font-size: 20px; }`

Pseudo Classes: Link Style

- Static (relatively)
  - Not visited link:
    ```
    a:link { color: green; }
    ```
  - Visited link:
    ```
    a:visited { color: red; }
    ```
- Dynamic
  - Mouse moved over:
    ```
    a:hover { … }
    ```
  - Link activation:
    ```
    a:active { … }
    ```
  - Keyboar focus:
    ```
    a:focus { … }
    ```

Typical Link Style

- Highlighted by color, underlines on moving mouse over
- It gets highlighed when activated, gets darker when visited

```
a { color: #aaf; text-decoration: none; }
a:link { }
a:visited { color: #339; }
a:hover { text-decoration: underline; }
a:active { text-decoration: underline; color: red; }
```

Example: Sample link.

Some tricks about links

- Element `<a>` always belongs to some pseudoclass
  - It's changing according to circumstances
- Rule `a { … }` has lower priority than the rules with a pseudoclass
  - Pseudoclass has the same specificity as a class
- The `a` rule cannot redefine the properties already defined for pseudoclasses

Pseudo Class :first-child

- The first child element of its parent element

```html
<ul class="menu">
<li>Blue item</li>
<li>Green item</li>
<li>Green item</li>
</ul>
```

```css
.menu li { color: green; }
.menu li:first-child { color: blue; }
```

- Similarly `last-child`, much more in CSS3

Pseudo Elements: Firtst Letters and Lines

- Element `:first-line`
  - Denotes the first line of an arbitrary element
  - E.g. `p:first-line` - first line of a paragraph
  - Depends on the window width, font size, …
- Element `:first-letter`
  - Denotes first letter of an element
  - E.g. `p:first-letter` - first letter of a paragraph

Example 1

```
p { text-transform: none; }
p:first-line { text-transform: uppercase; }
```

```
<p>First line of this line will
be in capital letters. Any text follows. Any text follows.
Any text follows.</p>
```

First line of this line will be in capital letters. Any text follows. Any text follows. Any text follows.

Example 2

```
p:first-letter {
    font-size: 200%;
    font-weight: bold;
    float: left;
}
```

```
<p>First letter will be large and floated
by the remaining text. Other text as normal. Other text as normal.
Other text as normal.</p>
```

First letter will be large and floated by the remaining text. Other text as normal. Other text as normal. Other text as normal.

Generated content

- Low support in browsers (missing in MSIE)
- Allows to generate a text that is not present in the document
- Pseudo-elements `:before` a `:after`
  - `:before` - add a text before the element contents
  - `:after` - after the text contents
- The `content:` property
  - The text string to be inserted

Example

```
div.remark:before {
    content: "Remark: ";
    font-weight: bold;
}
```

```
<div class="pozn">This rule doesn't always apply.</div>
```

This rule doesn't always apply.

To be continued…

CSS – Creating a complex layout